SLIDE 1

What the agents actually said to each other

703 messages across 147 attempts — every figure below emitted by one script over the raw logs: replication_messages.py

HOW IT IS DONE — `replication_messages.py`

Each message feature is a **literal string search**: we join an attempt's messages into one string and test for a marker — a filename token such as `loaders.py` for "names a file", a `?` for "asks a question", a code fence or `def name(` for "sends code". When an agent finishes, its work is saved as a **diff**: a machine-readable list of which files it changed and which line numbers it touched — so for every attempt that clashed, we open *both* agents' diffs and look for line ranges that overlap, which pinpoints the collision in the code. Checking those filenames against the conversation establishes the central claim below: the agents had already discussed the file they collided in.

① Measured from the message logs

Measure	Value	What the machine literally looks for
They name at least one **file**	**100%**	text contains *x*.py / .go / .rs …
Either agent asks a **question**	**9%**	text contains "?"
They exchange **actual code**	**3%**	text contains ``` or "def *name*(" — a lower bound
Messages per attempt (median)	**5**	count of messages in the log
Whole conversation (median)	**114s**	last timestamp minus first

② Measured from the code the agents wrote — the evidence the argument rests on

On the attempts that failed to combine	Value	How it is computed
Clashing attempts examined	**93**	the combine failed and both agents' work was saved
…both changed the **same file**	**100%**	compare the two file lists for a name in both
…that file was **named in the chat**	**100%**	search the conversation for that filename
…they changed the **same lines**	**90%**	compare the two line ranges for an overlap
…both rewrote the **same function**	**27%**	compare the function names each one edited

Each row compares the two agents' saved changes against each other.

One attempt, verbatim (it clashed)

agent 2 *"Suggest final combined order:* `(environment, value, attribute, default=None, separator='.', reverse=False)` *— just append your reverse param after mine."*

agent 1 *"my working copy doesn't show your separator changes… we appear to be in separate sandboxes. Since each of us submits our own work independently, this should be fine — no actual clobbering."*

Agent 2 spells out the correct combined line; agent 1 agrees; **neither types it**. One documented instance of the mechanism.

They agree on a description of the answer. But the combine is decided by the exact characters they type.

Scope. Every figure here is a literal count or a diff comparison, so each one is reproducible from the logs by rerunning the script. Scoring conversations for habits like "both said they'd add theirs last" would need a hand-written phrase list, which belongs to a separate coding study with its own reliability check.

SLIDE 2

Six sets of rules, and what each one tries to fix

Each rung forces the pair to agree on more of the final code than the rung below

- Same model, tasks, marking and automatic combine throughout — **only the cooperation rules change**, so any difference is caused by the rules.
- They form a ladder: agree on **nothing** at the bottom, on **the exact code** at the top.

	Rule set	They must agree on…	What it is trying to fix	Combined / both work
1	**control**	Nothing — no messaging at all	The baseline. If the talking is doing real work, removing it should hurt.	13% / 2%
2	**free-text**	Whatever they choose to say	The original condition, repeated here so the other five have something to beat.	21% / 3%
3	**semi-structured**	A filled-in form about their intent	Tests whether the problem was just **sloppiness**: forces every message to name its files and intent, or be rejected.	16% / 3%
4	**plan-handshake**	A split of the files, agreed up front	Fixes the missing hand-out of work: they must **agree who owns which file and wait for a reply** before typing anything.	20% / 10%
5	**designated-coder**	One owner per shared file	Accepts that a file both need cannot be split, so **only one agent writes it**; the other sends a written request instead.	18% / 58%
6	**coauthor-overlap**	**The exact merged code itself**	Goes after the clash directly: they agree the shared code word-for-word, then **both type the identical text**.	**78% / 69%**

Why these six, and why in this order

- Slide 1: clashes happen **inside** a file they already discussed, on the **same line**. Agreeing about *files* is too coarse to prevent them.
- So the result is **predictable before running anything**: rules 1–4 agree only at the level of files or intentions and should fail; only 5–6 reach the code itself.
- Rules 3 and 4 are the useful failures — **3**: they already named their files every time, so imposing structure on the message adds nothing. **4**: a real agreed split made the code more *correct*, and no easier to combine.

> **TAKEAWAY** *Each rung takes away one more thing the two agents can disagree about. The top rung takes away all of them.*

Result column = slide 3's figures, from `analyze.py` over the run logs.

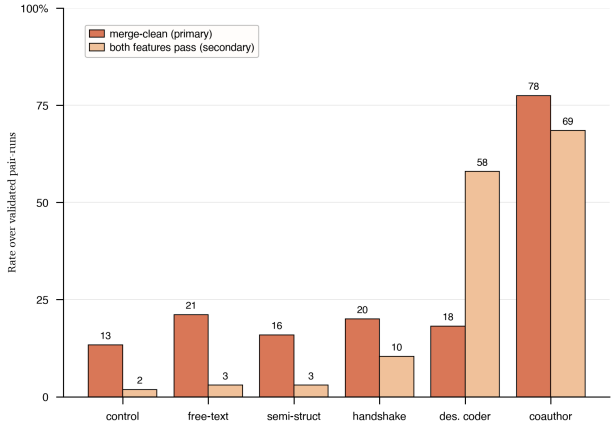
SLIDE 3

What happened

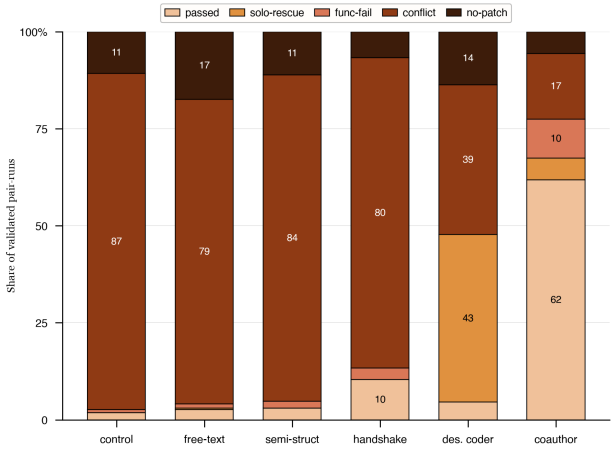
18 tasks chosen because the two features genuinely overlap · ~1,250 attempts · Claude Sonnet 5

READING THE NUMBERS — the two things being measured

Combined cleanly	The two sets of edits went together automatically. *Headline measure.*
Both features work	After combining, both features pass their tests. *Stricter measure.*
Clash	Both rewrote the same lines, so the combine refused.
One carried the pair	Both features work only because **one** agent's copy already contained both — not because the two combined.



How often each rule set worked. Taller is better. Five sit in the same low band; only coauthor-overlap breaks out.



How attempts failed. One bucket each. The clash band dominates everywhere except the last rule set.

Rule set	Combined cleanly	Both features work	Clashed
control	13%	2%	87%
free-text	21%	3%	79%
semi-structured	16%	3%	84%
plan-handshake	20%	10%	80%
designated-coder	18%	58%	39%
coauthor-overlap	**78%**	**69%**	**17%**

Key findings

- **Only one rule set worked.** Co-writing the shared code took clean combining **13%** → **78%** and both-features-working **2%** → **69%** — the only arm to beat the baseline once you account for having tried six things.
- **Talking more achieved nothing.** Tidier messages, or planning who owns which file, scored no better than **no messaging at all** — as slide 1 predicted.
- **It works by changing what gets typed.** The winner produced **26** byte-identical results; the other five produced **1** between them across 1,168 attempts.
- **Working code ≠ combinable code.** designated-coder reaches 58% both-working while clashing as often as the baseline — one agent quietly carried the pair.
- **Not solved.** The best rule set still clashes 17% of the time and breaks another 10%. One model, one setup, one style of automatic combine.

TAKEAWAY *Rules for cooperation only help when they constrain **what the agents type** — not how much they say to each other.*

Source. All numbers and both figures from `analyze.py` over `logs/`, frozen in `data/nano_study.json`; charts drawn by `figures.py`. "Beats the baseline" = significant under a Cochran–Mantel–Haenszel test stratified by task, Holm-corrected across the eight comparisons.